

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

GURAJADA VIZIANAGARAM

VIZIANAGARAM – 533003

ANDHRA PRADESH

BLOOD DONOR FINDER SYSTEM

By

LALAM PADMINI

(Regd. No. 24VV1A1236)

Under the supervision of

MR. W. ANIL

MRS. M. GEETHA MADHURI YADAV

Department of Information Technology

Vizianagaram – 533003



JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Regd.No : 24VV1A1236

CERTIFICATE

This is to certify that this is a bonafide record of practical work done by **LALAM PADMINI** of IIIrd B.Tech Ist Semester Class in Advance Java Programming Lab during the year 2026-27.

No.of Tasks Completed and Certified:

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: ADVANCED JAVA LAB **Subject Code: R233112PCO4P**

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112PCO4P	CO1	Develops JDBC applications to connect with a database and perform CRUD operations using Statement and PreparedStatement objects.
	CO2	Distinguishes Application Server, Web Container and JDBC, and understands J2EE web application architecture and deployment.
	CO3	Designs and implements server-side web components using the Servlet API.
	CO4	Develops dynamic web pages using JSP and implements components such as JSTL.
	CO5	Designs and develops web applications with the collaboration of Servlets, JSPs and the Spring framework.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112PCO4P	CO1	3	2	2	2	3	1	1	1	2	2	1	2	3	2	2
	CO2	2	2	2	1	2	1	1	1	2	2	1	2	3	2	2
	CO3	3	2	3	2	3	1	1	1	2	2	2	2	3	3	2
	CO4	3	2	3	2	3	1	1	1	2	2	2	2	3	3	2
	CO5	3	3	3	3	3	2	1	1	3	3	3	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-".

Signature of the Course Instructor

Blood Donor Finder System

Aim

To develop a web-based Blood Donor Finder System using HTML, CSS, JSP, Java Servlets, JDBC, and MySQL. The application helps users find available blood donors, create and respond to blood requests, track request status, and helps administrators manage users, donors, requests, and blood inventory.

Lab Exercise: Blood Donor Finder System

Objective: Develop a Blood Donor Finder System using Java Servlets, JSP, and MySQL. The system should allow users to register and login, add donor details, search available donors by blood group and city, create blood requests, respond to pending requests, view responses and notifications, and track request status. An administrator should be able to manage users, donors, requests, and blood inventory.

Instructions:

Database Setup:
Create a MySQL database named `blood_donor_system`. The project uses tables for users, donors, requests, donor responses, notifications, administrators, and blood inventory.

Login Page:
Create a login page for registered users and validate credentials through the login servlet. After successful login, store the user's identity in an HTTP session. Protected pages and servlets should check the session before allowing access. The project also has a separate administrator login.

Donor Operations:
Provide a form to add donor details such as blood group, age, gender, city, and availability. Allow users to search available donors using blood group and city.

Blood Request Operations:
Allow users to create blood requests with the blood group, number of units required, and city. Users can view their own requests, see pending requests created by other users, and respond to requests where eligible. Requesters can view the responses and status of their requests.

Administrator Operations:
Provide an administrator dashboard to manage users, donors, blood requests, responses, and blood inventory. The administrator can perform the status actions supported by the application.

User Interface:
Create JSP pages and HTML forms for login, dashboard navigation, donor registration, donor search, blood requests, pending requests, donor responses, notifications, profile, and administrator features. Display records in readable lists or cards.

Navigation:
Provide navigation from the user dashboard to donor and request operations, responses, notifications, and profile. Provide separate navigation for administrator operations.

Hardware and Software Requirements:

- **Hardware:** Desktop or laptop computer.
- **Operating System:** Windows.
- **Programming Language:** Java.
- **Frontend Technologies:** HTML, CSS, JSP.
- **Backend Technologies:** Java Servlets.
- **Database:** MySQL.
- **Database Connectivity:** JDBC.
- **Server:** Apache Tomcat 11.
- **IDE:** Eclipse IDE.

Working Principle:
The user accesses the application through a web browser. JSP pages provide the interface, and Java Servlets process user requests. JDBC connects the application to the MySQL database to retrieve or update information. The results are displayed on the appropriate JSP pages. HTTP sessions maintain the login state of users while they access protected application features.

Benefits of the System:

- Enables searching for available donors by blood group and city.
- Organizes blood requests and donor responses.
- Helps users track the status of their requests.
- Provides notifications for relevant updates.
- Allows administrators to manage donor records and blood inventory.

Project Overview

The Blood Donor Finder System is a web-based application developed using Java Servlets, JSP, JDBC, and MySQL. It provides a common platform for users who need blood and people who can respond to blood requests. Users can maintain donor information, search for available donors by blood group and city, create requests, respond to pending requests, and check responses and notifications. The administrator can manage user and donor records, review blood requests, and maintain blood inventory.

The application uses JSP for dynamic presentation, Java Servlets to process browser requests, and JDBC to communicate with the MySQL database. HTTP sessions maintain the logged-in user's identity as they navigate protected pages. The system is intended to make blood request handling more organized and to provide a central place to track request status and responses.

Objectives

1. Provide login and session-based access for registered users.
2. Allow users to add and maintain donor information.
3. Search available donors using blood group and city.
4. Allow users to create blood requests with the required blood group, units, and city.
5. Allow eligible users to accept or decline pending requests.
6. Allow requesters to view donor responses and track request status.
7. Provide notifications and unread-notification count.
8. Provide an administrator dashboard for user, donor, request, and inventory management.
9. Use JDBC and prepared statements for database operations.

Key Features

User Management and Authentication

- **User Login:** Authenticates registered users.
- **Session Management:** Uses `HttpSession` to maintain login state.
- **My Profile:** Provides access to the user's profile information.

- **Logout:** Ends the active user session.

Donor and Blood Request Management

- **Add Donor:** Saves donor details and availability.
- **Search Donor:** Finds available donors by blood group and city.
- **Create Blood Request:** Records the requested blood group, units, and city.
- **Pending Requests:** Lists pending requests created by other users, according to the application's response rules.
- **Donor Response:** Allows an eligible user to accept or decline a request.
- **My Requests:** Displays requests created by the logged-in user and their statuses.
- **Donor Responses:** Displays response details to the requester.
- **Notifications:** Displays user notifications and unread count.

Administrator Features

- **Admin Dashboard:** Provides access to administrative modules.
- **Manage Users:** Lists and manages registered users.
- **Manage Donors:** Lists and manages donor records.
- **Manage Requests and Responses:** Reviews requests and updates status using available actions.
- **Blood Inventory:** Adds or updates available units for blood groups.

Security Features

- Session checks are used by protected user features.
- `PreparedStatement` is used for SQL operations that receive input values.
- Server-side validation should be applied to form values before database updates.
- Administrator pages should be protected by administrator authentication and authorization.

System Architecture

The application follows an MVC-style separation of responsibilities.

- **Presentation Layer:** JSP pages, HTML forms, and CSS display pages and results.
- **Controller Layer:** Java Servlets receive HTTP requests, validate inputs and sessions, perform application flow, and forward or redirect responses.
- **Database Layer:** JDBC classes connect to MySQL and execute SQL queries.
- **Web Server:** Apache Tomcat runs the web application.

Technology Stack

- **Frontend:** HTML, CSS, JSP
- **Backend:** Java Servlets
- **Database Connectivity:** JDBC
- **Database:** MySQL
- **Server:** Apache Tomcat 11
- **IDE:** Eclipse IDE
- **Session Management:** `HttpSession`

Design Patterns

- **MVC-style separation:** JSP is used for presentation and Servlets handle requests and control flow.
- **JDBC data access:** A database connection utility and SQL statements are used to communicate with MySQL.
- **Session-based access control:** Session attributes identify the current logged-in user.

Project Structure

```
BloodDonorSystem
|
|-- Deployment Descriptor: BloodDonorSystem
|
|-- Java Resources
| |
| |-- src/main/java
| | |
| | |-- (default package)
| | | |-- DBConnection.java
| | | |-- LoginTest.java
| | | `-- TestDB.java
| | |
| | |-- com.blooddonor.controller
| | | |-- AddDonorServlet.java
| | | |-- AdminDashboardServlet.java
| | | |-- AdminLoginServlet.java
| | | |-- AdminLogoutServlet.java
| | | |-- BloodInventoryServlet.java
| | | |-- CreateRequestServlet.java
| | | |-- DonorResponseServlet.java
| | | |-- LoginServlet.java
| | | |-- LogoutServlet.java
| | | |-- ManageDonorsServlet.java
| | | |-- ManageRequestsServlet.java
| | | |-- ManageResponsesServlet.java
| | | |-- ManageUsersServlet.java
| | | |-- MarkNotificationReadServlet.java
| | | |-- MyDonorResponsesServlet.java
| | | |-- MyRequestsServlet.java
| | | |-- NotificationServlet.java
| | | |-- ProfileServlet.java
| | | |-- RegisterServlet.java
| | | |-- RespondToRequestServlet.java
| | | |-- SearchDonorServlet.java
| | | |-- UpdateInventoryServlet.java
| | | |-- UpdateRequestStatusServlet.java
| | | `-- ViewPendingRequestsServlet.java
| | |
| | |-- com.blooddonor.model
| | | |-- BloodInventory.java
| | | |-- BloodRequest.java
| | | |-- Donor.java
| | | |-- DonorResponse.java
| | | |-- Notification.java
| | | `-- User.java
| | |
| | `-- com.blooddonor.util
| | `-- DBConnection.java
| |
| `-- Libraries
|
|-- Referenced Libraries
|
|-- build
|
`-- src
    |
    `-- main
        |
```

```
    `-- webapp
        |
        |-- META-INF
        |
        |-- WEB-INF
        | |
        | |-- lib
        | | `-- mysql-connector-j-26.7.0.jar
        | |
        | `-- web.xml
        |
        |-- add-donor.jsp
        |-- admin-dashboard.jsp
        |-- admin-login.jsp
        |-- blood-inventory.jsp
        |-- create-request.jsp
        |-- dashboard.jsp
        |-- donor-response.jsp
        |-- donor-results.jsp
        |-- index.jsp
        |-- login.jsp
        |-- manage-donors.jsp
        |-- manage-requests.jsp
        |-- manage-responses.jsp
        |-- manage-users.jsp
        |-- my-donor-responses.jsp
        |-- my-requests.jsp
        |-- notifications.jsp
        |-- pending-requests.jsp
        |-- profile.jsp
        |-- register.jsp
        `-- search-donor.jsp
```

Key Components

1. Model / Database Entities

The database tables used in this project represent the following entities.

Table	Purpose
`users`	Registered user account and profile information.
`donors`	Donor details, blood group, age, gender, city, and availability.
`requests`	Blood request details, requester, blood group, required units, city, status, and request date.
`donor_responses`	Responses linked to blood requests and donor records.
`notifications`	Notifications associated with users and read/unread status.
`admin`	Administrator login information.
`blood_inventory`	Blood-group inventory and available units.

2. Database Connection Utility

The `DBConnection.java` class is used to establish a connection between the Java application and the MySQL database. It is used by the application components that need to execute database operations.

3. Controller Layer (Servlets)

Servlets receive HTTP requests and process the selected operation. The project shows the following controller classes:

- `LoginServlet.java` and `RegisterServlet.java` – user login and registration.
- `AddDonorServlet.java` and `SearchDonorServlet.java` – donor registration and search.
- `CreateRequestServlet.java`, `ViewPendingRequestsServlet.java`, and `MyRequestsServlet.java` – blood request operations.
- `DonorResponseServlet.java`, `RespondToRequestServlet.java`, and `MyDonorResponsesServlet.java` – request response handling and display.
- `NotificationServlet.java` and `MarkNotificationReadServlet.java` – notification features.
- `ProfileServlet.java` – profile operations.
- `AdminLoginServlet.java`, `AdminDashboardServlet.java`, and `AdminLogoutServlet.java` – administrator authentication and navigation.
- `ManageUsersServlet.java`, `ManageDonorsServlet.java`, `ManageRequestsServlet.java`, and `ManageResponsesServlet.java` – administrative management.
- `BloodInventoryServlet.java` and `UpdateInventoryServlet.java` – blood inventory features.
- `UpdateRequestStatusServlet.java` – request status updates.

4. Presentation Layer (JSP Pages)

JSP pages provide forms and dynamic displays for users and administrators. The relevant pages include the login and registration pages, user dashboard, donor form and search results, request pages, donor response pages, notification page, profile page, admin dashboard, and management pages.

5. Servlets (Controllers)

Servlets are an important component of the Blood Donor Finder System. They act as controllers in the application and handle HTTP requests received from the user's browser. Each servlet processes a specific operation, such as user login, registration, donor management, blood request processing, notifications, or administrator operations.

The servlets communicate with the MySQL database using JDBC. They use SQL queries and PreparedStatement objects to insert, retrieve, update, and manage application data. Based on the operation performed, the servlet forwards the request to a JSP page or redirects the user to another page.

The application uses HttpSession to maintain the login state of users and to restrict access to protected features. Servlet annotations are used to map servlet classes to their respective URL patterns.

DBConnection.java:

```
import java.sql.Connection;
import java.sql.DriverManager;
public class DBConnection {
    public static Connection getConnection() {
        Connection con = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/blood_donor_system",
                "root",
                "p@1236" // <-- mee password change cheyyi
            );
            System.out.println("Database Connected Successfully");
        } catch (Exception e) {
            e.printStackTrace();
        }
        return con;
    }
    public static void main(String[] args) {
        getConnection();
    }
}
```

RegisterServlet.java:

```
package com.blooddonor.controller;
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import com.blooddonor.util.DBConnection;
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @Override
    protected void doPost(
            HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String phone = request.getParameter("phone");
        if (name == null || email == null || password == null || phone == null || name.trim().isEmpty() ||
             email.trim().isEmpty() || password.trim().isEmpty() || phone.trim().isEmpty()) {
            response.sendRedirect( "register.jsp?error=empty" );
            return;
        }
        phone = phone.trim();
        if (!phone.matches("\\d{10}")) {
            response.sendRedirect( "register.jsp?error=phone" );
            return;
        }
        Connection con = null;
        try {
            con = DBConnection.getConnection();
            String checkSql="SELECT user_id"+"FROM users"+"WHERE email = ?";
            PreparedStatement checkPs = con.prepareStatement( checkSql );
            checkPs.setString( 1, email.trim() );
            ResultSet checkRs = checkPs.executeQuery();
            if (checkRs.next()) {
                checkRs.close();
                checkPs.close();
                con.close();
                response.sendRedirect( "register.jsp?error=email" );
                return;
            }
            checkRs.close();
            checkPs.close();
            String sql =
                    "INSERT INTO users " +
                    "(name, email, password, phone, role) " +
                    "VALUES (?, ?, ?, ?, 'USER')";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setString( 1, name.trim() );
            ps.setString( 2, email.trim() );
            ps.setString( 3, password );
            ps.setString( 4, phone);
            int rows = ps.executeUpdate();
            ps.close();
            con.close();
            if (rows == 1) {
                response.sendRedirect( "login.jsp?success=registered" );
            } else {
                response.sendRedirect( "register.jsp?error=failed" );
            }
        } catch (Exception e) {
            e.printStackTrace();
            try {
                if (con != null &&
```

```
                !con.isClosed()) {
                con.close();
            }
        } catch (Exception ex) {
            ex.printStackTrace();
        }
        response.sendRedirect( "register.jsp?error=database" );
    }
  }
}
```

LoginServlet.java:

```
package com.blooddonor.controller;
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import com.blooddonor.util.DBConnection;
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
   private static final long serialVersionUID = 1L;
   protected void doPost(
           HttpServletRequest request,
           HttpServletResponse response)
           throws ServletException, IOException {
       String email = request.getParameter("email");
       String password = request.getParameter("password");
       String sql = "SELECT * FROM users WHERE email=? AND password=?";
       try {
           Connection con = DBConnection.getConnection();
           PreparedStatement ps = con.prepareStatement(sql);
           ps.setString(1, email);
           ps.setString(2, password);
           ResultSet rs = ps.executeQuery();
           if (rs.next()) {
               HttpSession session = request.getSession();
               session.setAttribute("userId",rs.getInt("user_id"));
               session.setAttribute("userName",rs.getString("name"));
               session.setAttribute("userEmail",rs.getString("email"));
               response.sendRedirect("dashboard.jsp");
           } else {
               response.sendRedirect("login.jsp?error=1");
           }
           rs.close();
           ps.close();
           con.close();
       } catch (Exception e) {
           e.printStackTrace();
           response.sendRedirect("login.jsp?error=1");
       }
   }
}
```

AddDonorServlet.java:

```
package com.blooddonor.controller;
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import com.blooddonor.util.DBConnection;
@WebServlet("/addDonor")
public class AddDonorServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    protected void doPost(
            HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect("login.jsp");
            return;
        }
        int userId = (Integer) session.getAttribute("userId");
        String bloodGroup = request.getParameter("bloodGroup");
        int age = Integer.parseInt( request.getParameter("age") );
        String gender = request.getParameter("gender");
        String city = request.getParameter("city");
        String sql = "INSERT INTO donors " +
            "(user_id, blood_group, age, gender, city, availability) " +
            "VALUES (?, ?, ?, ?, ?, ?)";
        try {
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, userId);
            ps.setString(2, bloodGroup);
            ps.setInt(3, age);
            ps.setString(4, gender);
            ps.setString(5, city);
            ps.setBoolean(6, true);
            int rows = ps.executeUpdate();
            ps.close();
            con.close();
            if (rows > 0) {
                response.sendRedirect( "dashboard.jsp?success=donor" );
            } else {
                response.sendRedirect( "add-donor.jsp?error=1" );
            }
        } catch (Exception e) {
            e.printStackTrace();
            response.sendRedirect( "add-donor.jsp?error=1" );
        }
    }
}
```

SearchDonorServlet.java:

```
package com.blooddonor.controller;
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;
import java.util.List;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import com.blooddonor.util.DBConnection;
@WebServlet("/searchDonor")
public class SearchDonorServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @Override
    protected void doGet(
            HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect("login.jsp");
            return;
        }
        String bloodGroup = request.getParameter("bloodGroup");
        String city = request.getParameter("city");
        String availability = request.getParameter("availability");
        if (bloodGroup == null) {
            bloodGroup = "";
        }
        if (city == null) {
            city = "";
        }
        if (availability == null || availability.trim().isEmpty()) {
            availability = "AVAILABLE";
        }
        bloodGroup = bloodGroup.trim();
        city = city.trim();
        availability = availability.trim().toUpperCase();
        List<String[]> donors = new ArrayList<>();
        List<String[]> inventory = new ArrayList<>();
        Connection con = null;
        try {
            con = DBConnection.getConnection();
            StringBuilder sql = new StringBuilder();
            sql.append( "SELECT " +"d.donor_id, " +"u.name, " +"u.email, " +"u.phone, " +"d.blood_group, "
                 +"d.age, " +"d.gender, " + "d.city, " +"d.availability " +"FROM donors d "+"JOIN users u" +
                 "ON d.user_id = u.user_id " +"WHERE 1=1 " );
            List<Object> parameters = new ArrayList<>();
            if (!bloodGroup.isEmpty()) {
                sql.append( "AND d.blood_group = ? " );
                parameters.add( bloodGroup );
            }
            if (!city.isEmpty()) {
                sql.append( "AND LOWER(d.city) = LOWER(?) " );
                parameters.add( city );
            }
```

```
if ("AVAILABLE".equals(
        availability)) {
    sql.append( "AND d.availability = TRUE " );
}
sql.append( "ORDER BY d.blood_group, u.name" );
PreparedStatement ps =con.prepareStatement( sql.toString() );
for (int i = 0; i < parameters.size(); i++) {
    ps.setObject( i + 1, parameters.get(i) );
}
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    String phone = rs.getString("phone");
    if (phone == null) {
        phone = "";
    }
    String[] donor = {
        // 0 - Donor ID
        String.valueOf(rs.getInt("donor_id") ),
        // 1 - Name
        rs.getString("name"),
        // 2 - Email
        rs.getString("email"),
        // 3 - Phone
        phone,
        // 4 - Blood Group
        rs.getString( "blood_group" ),
        // 5 - Age
        String.valueOf( rs.getInt("age") ),
        // 6 - Gender
        rs.getString("gender"),
        // 7 - City
        rs.getString("city"),
        // 8 - Availability
        String.valueOf( rs.getBoolean( "availability" ) )
    };
    donors.add(donor);
}
rs.close();
pc.close();
String inventorySql;
if (!bloodGroup.isEmpty()) {
    inventorySql = "SELECT inventory_id, " +"blood_group, " + "units_available, " + "
         last_updated " +"FROM blood_inventory " + "WHERE blood_group = ? " +"ORDER BY
         blood_group";
} else {
    inventorySql = "SELECT inventory_id, " + "blood_group, " + "units_available, " + "
         last_updated " +
        "FROM blood_inventory " + "ORDER BY blood_group";
}
PreparedStatement inventoryPs =
con.prepareStatement( inventorySql );
if (!bloodGroup.isEmpty()) {
    inventoryPs.setString( 1, bloodGroup );
}
ResultSet inventoryRs = inventoryPs.executeQuery();
while (inventoryRs.next()) {
    String[] item = {
        // 0 - Inventory ID
        String.valueOf( inventoryRs.getInt( "inventory_id" ) ),
        // 1 - Blood Group
        inventoryRs.getString( "blood_group" ),
        // 2 - Units Available
        String.valueOf(inventoryRs.getInt("units_available") ),
```

```
                // 3 - Last Updated
            String.valueOf(inventoryRs.getTimestamp("last_updated"))
            };
            inventory.add(item);
        }
        inventoryRs.close();
        inventoryPs.close();
        int totalInventoryUnits = 0;
        String totalInventorySql;
        if (!bloodGroup.isEmpty()) {
            totalInventorySql = "SELECT COALESCE(" + "SUM(units_available), 0" + ") AS total " + "FROM
                 blood_inventory " + "WHERE blood_group = ?";
        } else {
            totalInventorySql = "SELECT COALESCE(" + "SUM(units_available), 0" + ") AS total " + "FROM
                 blood_inventory";
        }
        PreparedStatement totalPs = con.prepareStatement(totalInventorySql );
        if (!bloodGroup.isEmpty()) { totalPs.setString( 1, bloodGroup); }
        ResultSet totalRs = totalPs.executeQuery();
        if (totalRs.next()) { totalInventoryUnits = totalRs.getInt("total"); }
        totalRs.close();
        totalPs.close();
        int availableDonors = 0;
        String donorCountSql = "SELECT COUNT(*) AS total " + "FROM donors d " + "WHERE d.availability =
              TRUE ";
        List<Object> donorCountParameters = new ArrayList<>();
        if (!bloodGroup.isEmpty()) { donorCountSql += "AND d.blood_group = ? ";
            donorCountParameters.add( bloodGroup );
        }
        if (!city.isEmpty()) {
            donorCountSql += "AND LOWER(d.city) = LOWER(?) ";
            donorCountParameters.add( city );
        }
        PreparedStatement donorCountPs =con.prepareStatement( donorCountSql );
        for (int i = 0;i < donorCountParameters.size();i++) {
            donorCountPs.setObject(i + 1,donorCountParameters.get(i));
        }
        ResultSet donorCountRs = donorCountPs.executeQuery();
        if (donorCountRs.next()) { availableDonors = donorCountRs.getInt("total"); }
        donorCountRs.close();
        donorCountPs.close();
        con.close();
        request.setAttribute( "donors", donors );
        request.setAttribute( "bloodGroup", bloodGroup );
        request.setAttribute( "city", city );
        request.setAttribute( "availability", availability );
        request.setAttribute( "inventory", inventory );
        request.setAttribute( "totalInventoryUnits", totalInventoryUnits);
        request.setAttribute( "availableDonors", availableDonors );
        request.getRequestDispatcher("donor-results.jsp").forward(
            request,response);
      } catch (Exception e) {
        e.printStackTrace();
        try {
            if (con != null && !con.isClosed()) { con.close(); }
        } catch (Exception ex) {
            ex.printStackTrace();
        }
        response.sendRedirect( "search-donor.jsp?error=1" );
      }
   }
}
```

CreateRequestServlet.java:

```
package com.blooddonor.controller;
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import com.blooddonor.util.DBConnection;
@WebServlet("/createRequest")
public class CreateRequestServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @Override
    protected void doPost( HttpServletRequest request, HttpServletResponse response) throws
         ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            response.sendRedirect("login.jsp");
            return;
        }
        int userId = (Integer) session.getAttribute("userId");
        String bloodGroup = request.getParameter("bloodGroup");
        String unitsValue = request.getParameter("units");
        String city = request.getParameter("city");
        String emergencyValue = request.getParameter("emergency");
        if (bloodGroup == null || bloodGroup.trim().isEmpty() || unitsValue == null || unitsValue.trim().
             isEmpty() || city == null || city.trim().isEmpty()) {
            response.sendRedirect( "create-request.jsp?error=empty" );
            return;
        }
        int units;
        try {
            units = Integer.parseInt(unitsValue);
        } catch (NumberFormatException e) {
            response.sendRedirect("create-request.jsp?error=invalidUnits");
            return;
        }
        if (units <= 0) {
            response.sendRedirect("create-request.jsp?error=invalidUnits");
            return;
        }
        boolean emergency = "YES".equalsIgnoreCase(emergencyValue);
        Connection con = null;
        try {
            con = DBConnection.getConnection();
            String sql = "INSERT INTO requests " + "(requester_id, blood_group, " + "units_required, city,
                 status, emergency) " + "VALUES (?, ?, ?, ?, 'PENDING', ?)";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, userId);
            ps.setString( 2, bloodGroup.trim() );
            ps.setInt( 3, units );
            ps.setString( 4, city.trim() );
            ps.setBoolean( 5, emergency );
            int rows = ps.executeUpdate();
            ps.close();
            con.close();
            if (rows == 1) { response.sendRedirect("create-request.jsp?success=created"); }
            else { response.sendRedirect("create-request.jsp?error=failed"); }
```

```
        } catch (Exception e) {
            e.printStackTrace();
            try {
                if (con != null && !con.isClosed()) {
                    con.close();
                }
            } catch (Exception ex) {
                ex.printStackTrace();
            }
            response.sendRedirect( "create-request.jsp?error=database" );
        }
    }
}
```

AdminLoginServlet.java:

```
package com.blooddonor.controller;
import java.io.IOException;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import com.blooddonor.util.DBConnection;
@WebServlet("/adminLogin")
public class AdminLoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @Override
    protected void doPost( HttpServletRequest request, HttpServletResponse response) throws
         ServletException, IOException {
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String sql = "SELECT admin_id, name, email " + "FROM admin " + "WHERE email = ? AND password = ?";
        try { Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setString(1, email);
            ps.setString(2, password);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) { HttpSession session = request.getSession();
                session.setAttribute( "adminId", rs.getInt("admin_id") );
                session.setAttribute( "adminName", rs.getString("name") );
                session.setAttribute( "adminEmail", rs.getString("email"));
                response.sendRedirect( "adminDashboard" ); }
            else {response.sendRedirect( "admin-login.jsp?error=invalid");}
            rs.close();
            ps.close();
            con.close();
        } catch (Exception e) { e.printStackTrace(); response.sendRedirect( "admin-login.jsp?error=database
             " ); }
    }
}
```

JSP Codes:

register.jsp:

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Register - Blood Donor Finder</title>
<style>
* { box-sizing: border-box; }
body { margin: 0;
 font-family: Arial, sans-serif;
 min-height: 100vh;
 background: linear-gradient ( 135deg, #8e0e00, #c31432, #240b36 );
 display: flex;
 justify-content: center;
 align-items: center;
}
.register-card {
 width: 90%;
 max-width: 480px;
 background: white;
 padding: 38px;
 border-radius: 25px;
 box-shadow: 0 15px 40px rgba(0,0,0,0.3);
}
h1 {
 text-align: center;
 color: #8e0e00;
 margin-bottom: 10px;
}
.subtitle {
text-align: center;
color: #777;
margin-bottom: 25px;
}
.form-group {
margin-bottom: 18px;
}
label {
display: block;
margin-bottom: 7px;
color: #555;
font-weight: bold;
font-size: 14px;
}
input {
width: 100%;
padding: 12px 14px;
border:1px solid #ddd;
border-radius: 10px;
font-size: 15px;
outline: none;
}
input:focus {
border-color: #c31432;
box-shadow:0 0 0 3px rgba(195,20,50,0.1);
}
```

```
.register-btn {
 width: 100%;
 padding: 13px;
 border: none;
 border-radius: 25px;
 background: linear-gradient( 135deg, #8e0e00, #c31432 );
 color: white;
 font-size: 16px;
 font-weight: bold;
 cursor: pointer;
}
.login-link {
    text-align: center;
    margin-top: 20px;
    color: #777;
}
.login-link a {
    color: #c31432;
    text-decoration: none;
    font-weight: bold;
}
.message {
    padding: 12px;
    margin-bottom: 18px;
    border-radius: 10px;
    text-align: center;
    font-weight: bold;
}
.error {
    background: #f8d7da;
    color: #721c24;
    border: 1px solid #f5c6cb;
}
</style>
</head>
<body>
<div class="register-card">
    <h1> Create Account </h1>
    <p class="subtitle"> Register with your contact details </p>
<% String error = request.getParameter("error");
  if ("empty".equals(error)) {
%>
 <div class="message error"> Please fill all fields. </div>
<%
    }
    if ("phone".equals(error)) {
%>
    <div class="message error"> Enter a valid 10-digit phone number. </div>

<%
    }
    if ("email".equals(error)) {
%>
<div class="message error"> Email is already registered </div>

<%
    }
    if ("failed".equals(error)) {
%>
        <div class="message error"> Registration failed. </div>

<%
    }
```

```
    if ("database".equals(error)) {
%>
        <div class="message error"> Database error occurred. </div>

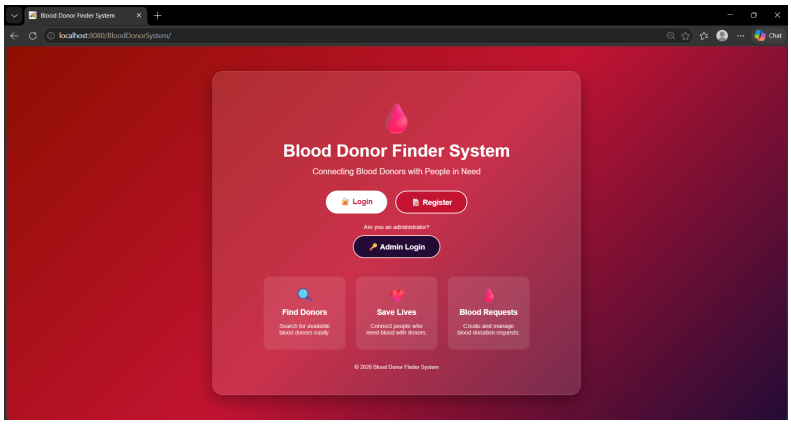
<%
    }
%>
    <form action="register" method="post">
        <div class="form-group">
            <label> Full Name </label>
            <input type="text" name="name" placeholder="Enter your name" required>
        </div>
        <div class="form-group">
            <label> Email </label>
            <input type="email" name="email" placeholder="Enter your email" required>
        </div>
        <div class="form-group">
            <label> Phone Number </label>
            <input type="tel" name="phone" placeholder="Enter 10-digit phone number" maxlength="10" pattern=
                "[0-9]{10}" required>
        </div>
        <div class="form-group">
            <label> Password </label>
            <input type="password" name="password" placeholder="Enter password" required>
        </div>
        <button type="submit" class="register-btn"> Create Account </button>
    </form>
    <div class="login-link"> Already have an account? <a href="login.jsp"> Login </a> </div>
</div>
</body>
</html>
```

login.jsp:

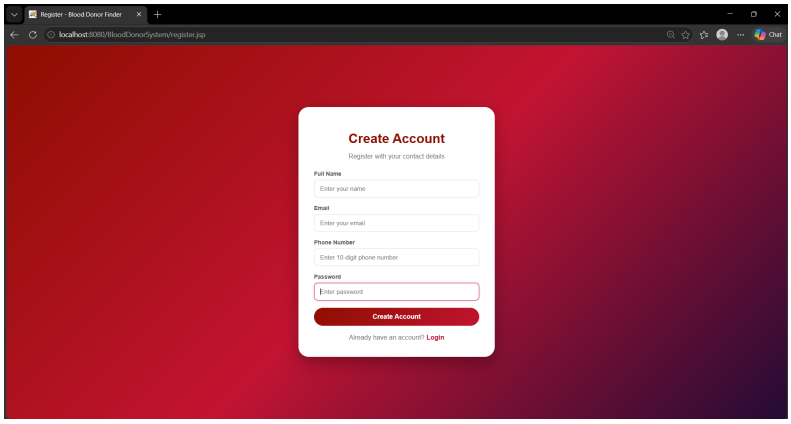
```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Blood Donor Finder - Login</title>
<style>
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #8e0e00, #1f1c18);
    height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}
.login-box {
    width: 400px;
    padding: 35px;
    background: white;
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.4);
}
h1 {
    text-align: center;
```

```
    color: #d90429;
}
input {
    width: 100%;
    padding: 12px;
    margin: 10px 0;
    border: 1px solid #ddd;
    border-radius: 8px;
    box-sizing: border-box;
}
button {
    width: 100%;
    padding: 12px;
    background: #d90429;
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    font-size: 16px;
}
button:hover {
    background: #a90321;
}
.register-link {
    text-align: center;
    margin-top: 15px;
}
.register-link a {
    color: #d90429;
    text-decoration: none;
}
.error {
    text-align: center;
    color: red;
    margin-bottom: 10px;
}
</style>
</head>
<body>
<div class="login-box">
    <h1> Blood Donor Finder</h1>
    <h3 style="text-align:center;"> Login </h3>
    <% String error = request.getParameter("error");
        if ("1".equals(error)) {
    %>
        <div class="error"> Invalid email or password! </div>
    <%
        }
    %>
    <form action="login" method="post">
        <input type="email" name="email" placeholder="Enter Email" required>
        <input type="password" name="password" placeholder="Enter Password" required>
        <button type="submit"> Login </button>
    </form>
    <div class="register-link">
        Don't have an account?
        <a href="register.jsp"> Register </a>
    </div>
</div>
</body>
</html>
```

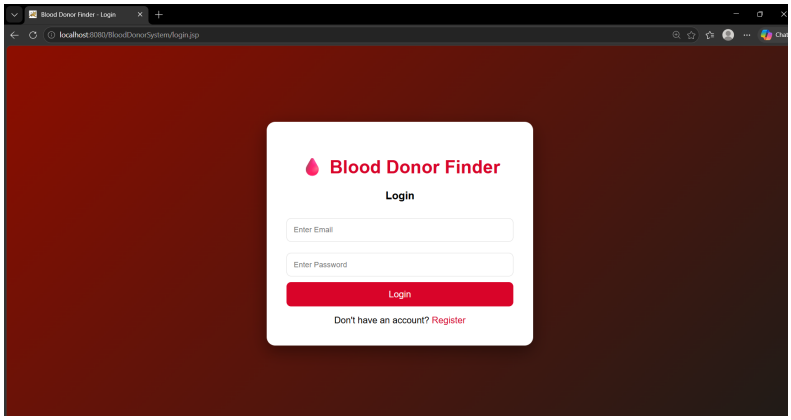
Outputs:



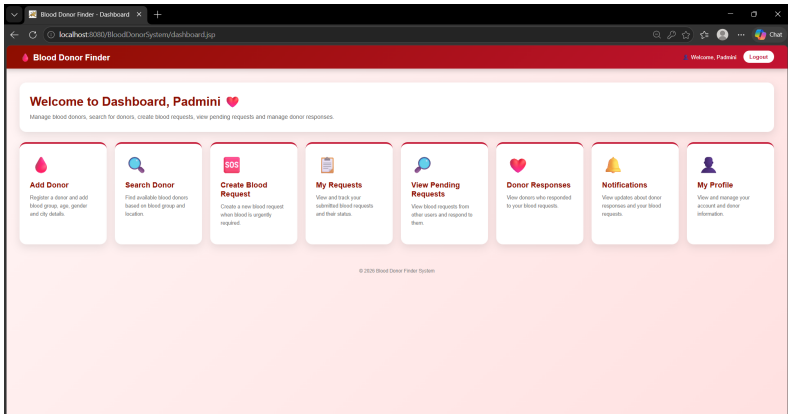
Welcome Page



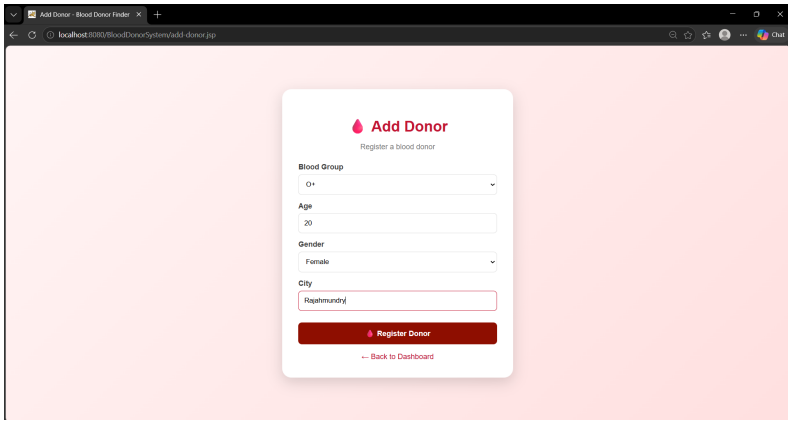
Register Page



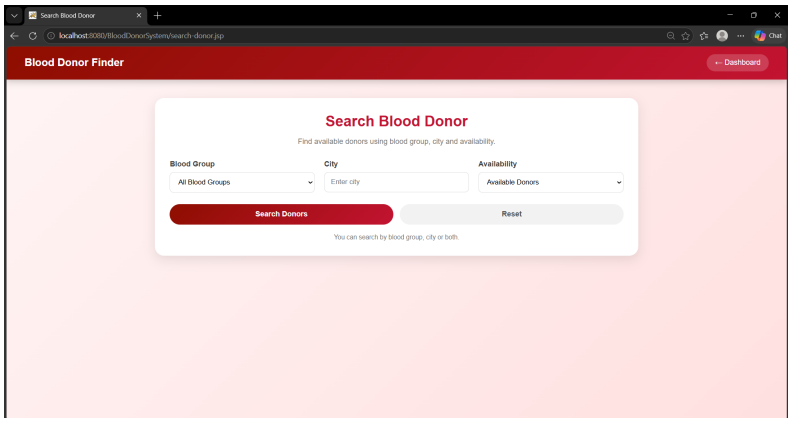
Login Page



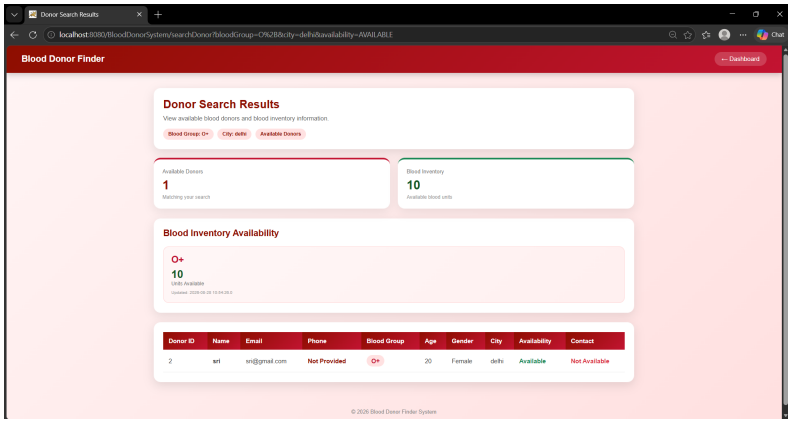
User Dashboard Page



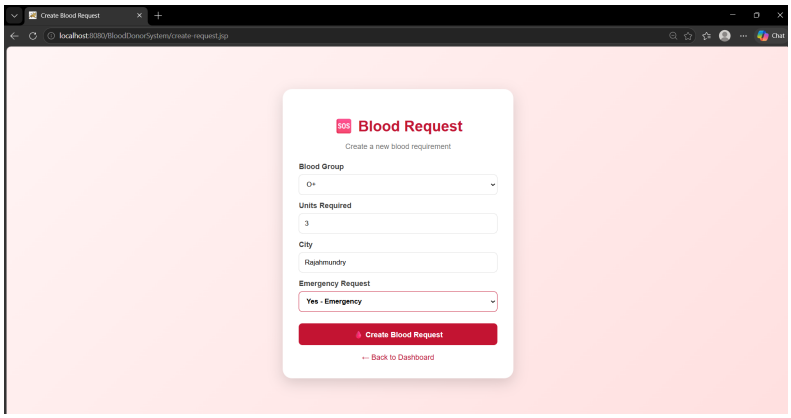
Add Blood Donor Page



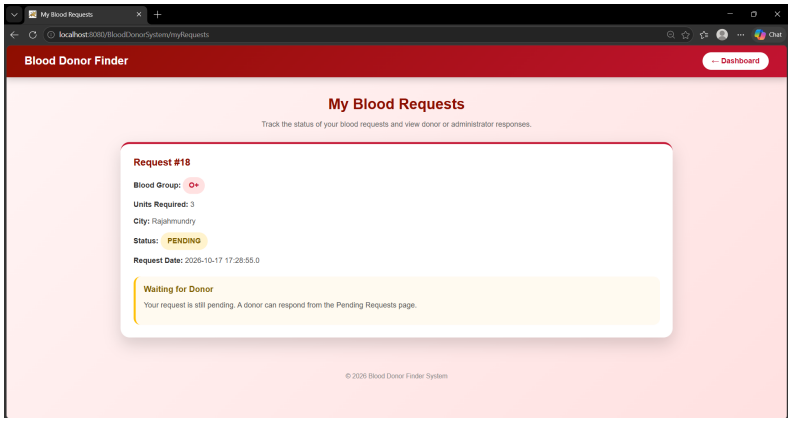
Search Blood Donor Page



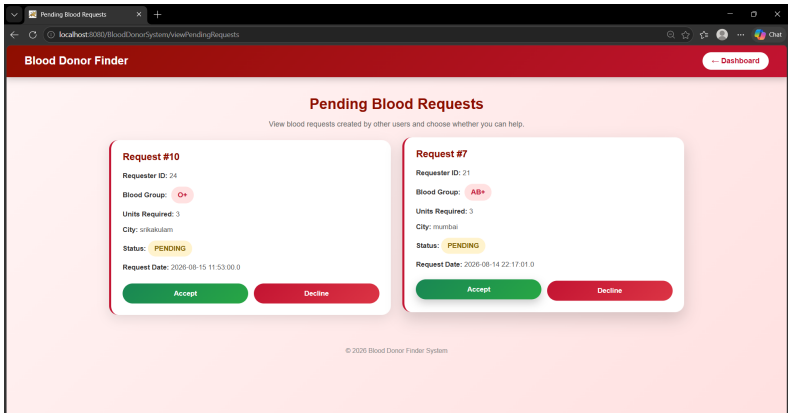
Search Blood Donor Result Page



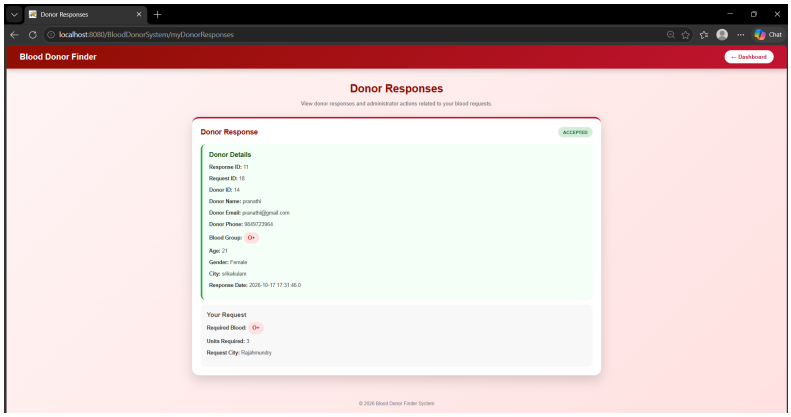
Create Blood Request Page



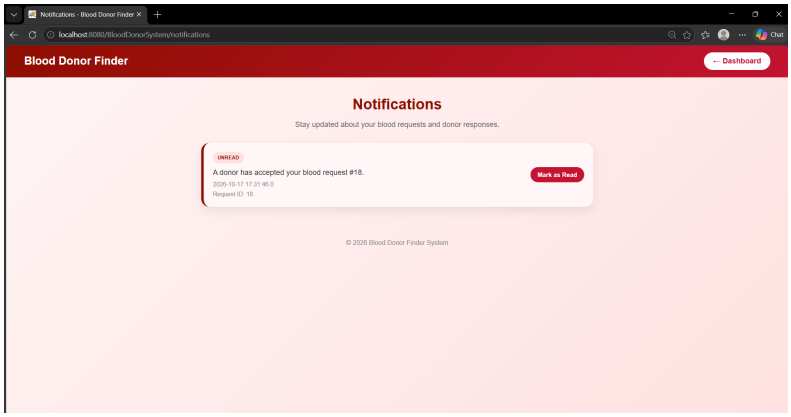
My Blood Requests Page



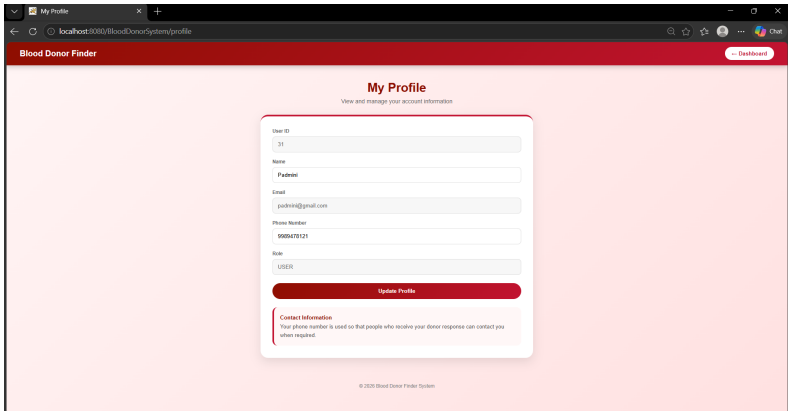
View Pending Blood Requests Page



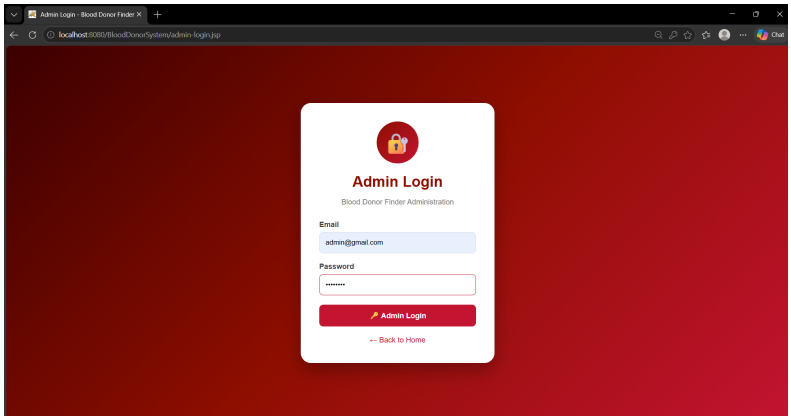
Donor Responses Page



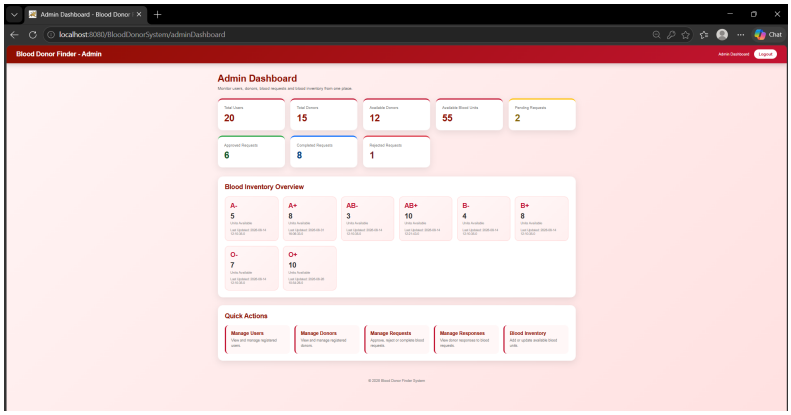
Notifications Page



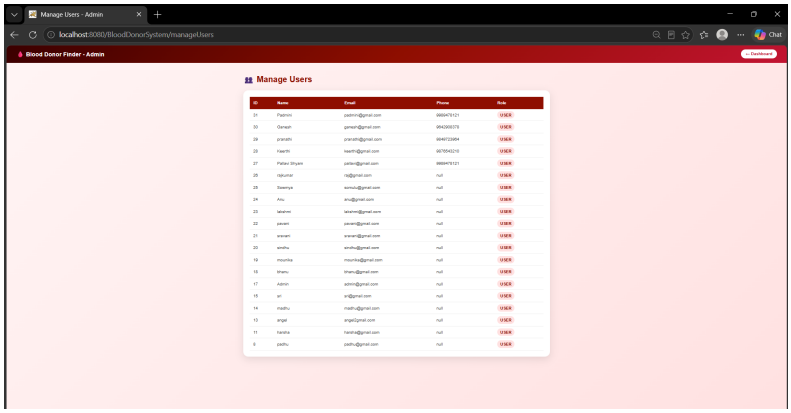
Profile Page



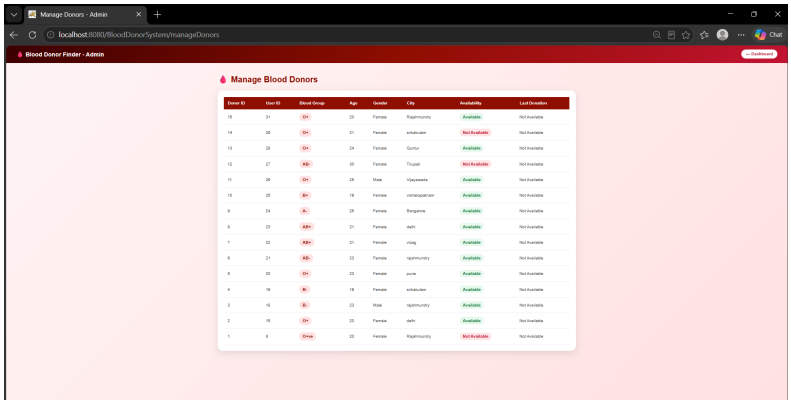
Admin Login Page



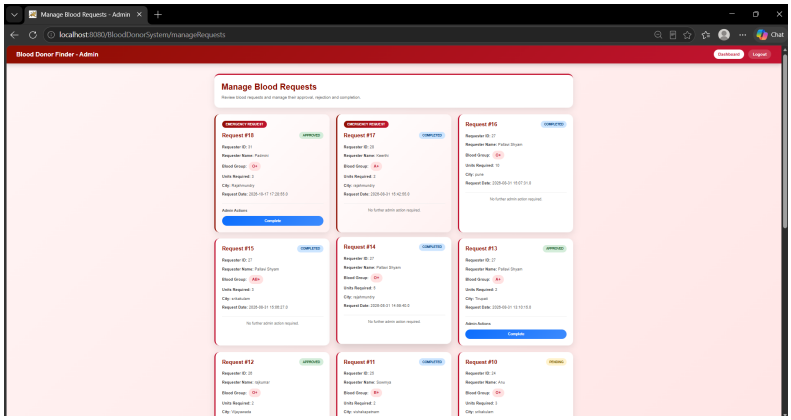
Admin Dashboard Page



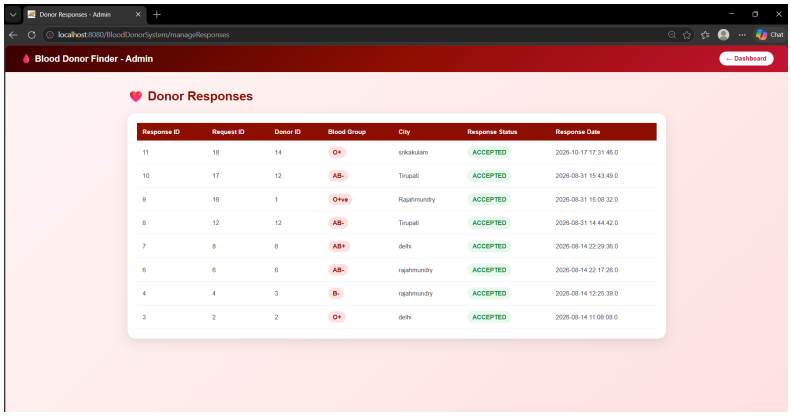
Admin Manage Users Page



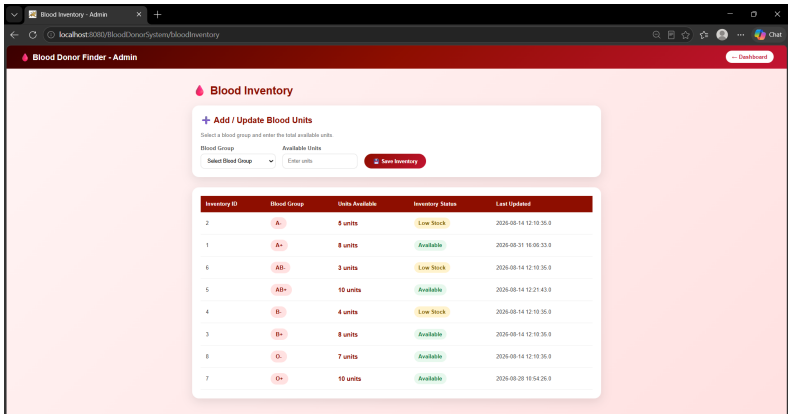
Admin Manage Donors Page



Admin Manage Requests Page



Admin Manage Responses Page



Admin Blood Inventory Page

Conclusion

The Blood Donor Finder System is a web-based application developed using Java, JSP, Servlets, JDBC, and MySQL. The main objective of this project is to provide a centralized platform for searching blood donors and managing blood requests efficiently.

The application allows users to register, log in, add donor details, search for available donors based on blood group and city, create blood requests, respond to pending requests, and track request status. The notification feature helps users view relevant updates related to their activities.

The administrator module provides facilities to manage registered users, donor information, blood requests, and blood inventory. JDBC and MySQL enable the application to store and retrieve information from the database, while Java Servlets process client requests and JSP pages display dynamic content.

The project also demonstrates important Advanced Java concepts, including Servlets, JSP, JDBC, PreparedStatement, ResultSet, HttpSession, HTTP request and response handling, and database connectivity. Apache Tomcat is used to run the web application.

Overall, the Blood Donor Finder System provides an organized approach to donor searching and blood request management. It can be further enhanced by introducing email or SMS alerts, location-based donor searching, improved security, and additional reporting features.

References

1. Oracle, Java Documentation.
2. Jakarta Servlet Documentation.
3. Jakarta Server Pages (JSP) Documentation.
4. MySQL Reference Manual.
5. Apache Tomcat Documentation.